

ESTRUTURA DE DADOS · BSI · 2º SEMESTRE

Projetos com Listas Duplamente Encadeadas

Dois mini-projetos práticos, criativos e rápidos de implementar

Até 1h30
cada

Lista Duplamente
Encadeada

Baseados no dia a
dia

1

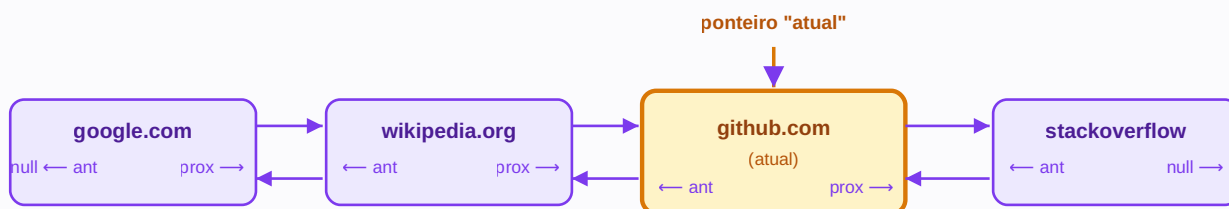
Histórico de Navegação

Simulando os botões "Voltar" e "Avançar" do navegador

Motivação: Todo mundo já clicou no botão "voltar" do navegador sem pensar em como ele funciona. Mas por trás dessa ação simples existe uma estrutura de dados perfeita: cada página visitada é um nó, e você precisa se mover para **trás** e para **frente** entre elas. É a lista duplamente encadeada em ação, escondida em algo que vocês usam todos os dias.

COMO FUNCIONA

Cada página visitada vira um **nó** da lista, contendo a URL (ou nome da página) e dois ponteiros: **anterior** e **proximo**. Um ponteiro extra, **atual**, indica em qual página o usuário está agora.



Estrutura do histórico: cada página é um nó com ligação para a anterior e para a próxima.

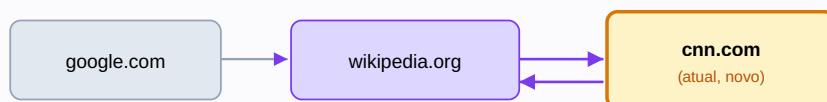
Quando o usuário clica em **Voltar**, o ponteiro **atual** passa a apontar para o nó **anterior**. Quando clica em **Avançar**, ele passa para o nó **proximo**. O ponto interessante (e que gera discussão em sala) é: **o que acontece se o usuário visita uma nova página estando "no meio" do histórico?**

Antes: usuário voltou até "wikipedia" e agora visita "cnn.com"



> github.com e stackoverflow serão descartados

Depois: novo nó "cnn.com" é inserido, o restante é perdido



Ao visitar uma nova página "no meio" do histórico, tudo que estava à frente é removido — igual acontece de verdade no navegador.

FUNCIONALIDADES OBRIGATÓRIAS

Operação	O que faz
<code>visitar(pagina)</code>	Adiciona uma nova página após a atual, descartando o que estava à frente
<code>voltar()</code>	Move o ponteiro atual para o nó anterior (se existir)
<code>avancar()</code>	Move o ponteiro atual para o próximo nó (se existir)
<code>mostrarHistorico()</code>	Exibe todas as páginas em ordem, destacando a atual

Sugestão de interface: pode ser um programa simples em modo texto (menu no terminal), com opções: [1] Visitar página, [2] Voltar, [3] Avançar, [4] Ver histórico, [5] Sair.

ROTEIRO SUGERIDO (1H30)

1. **(15 min)** Modelar a classe `Pagina` (nó) com atributos `url`, `anterior` e `proximo`.
2. **(15 min)** Implementar `visitar()`, incluindo o descarte do "futuro" quando necessário.
3. **(20 min)** Implementar `voltar()` e `avancar()`.
4. **(20 min)** Implementar `mostrarHistorico()` e o menu no terminal.
5. **(20 min)** Testar cenários: voltar até o início, avançar até o fim, visitar página nova no meio do histórico.

2

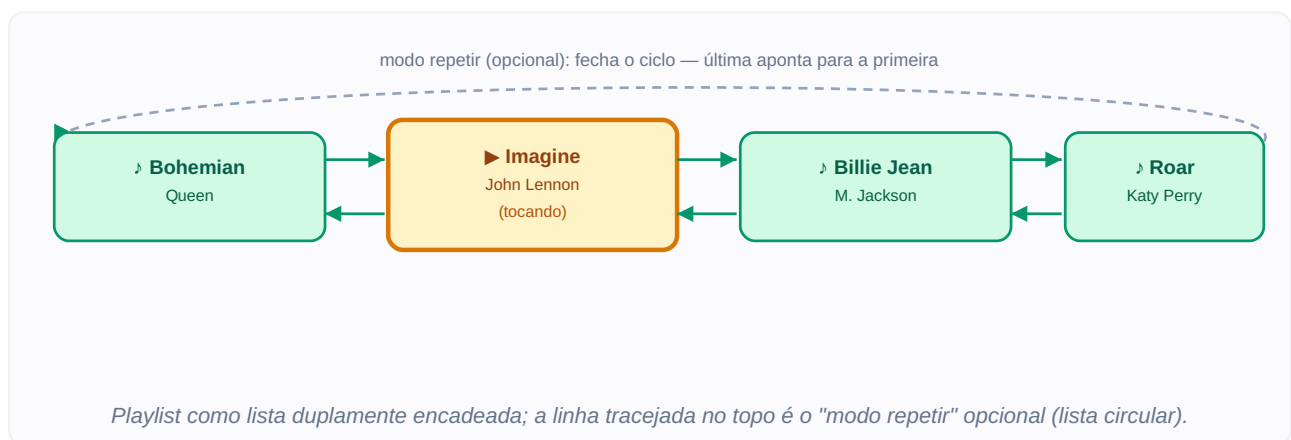
Playlist de Músicas

Um mini tocador com navegação entre faixas e modo repetir

Motivação: Todo mundo usa Spotify, YouTube Music ou similar. A pergunta que ninguém faz — mas que é ótima para aprender listas encadeadas — é: *como o player sabe qual é a "próxima" e a "anterior" música, e como ele consegue tocar em loop sem travar no final da lista?* Neste projeto, vocês vão construir a engrenagem por trás disso.

COMO FUNCIONA

Cada música é um nó contendo `titulo`, `artista`, e os ponteiros `anterior` e `proximo`. Um ponteiro `tocando` indica a faixa atual.



FUNCIONALIDADES OBRIGATÓRIAS

Operação	O que faz
<code>adicionarMusica(titulo, artista)</code>	Insere uma nova música no final da playlist
<code>removerMusica(titulo)</code>	Remove uma música específica, religando os vizinhos
<code>proxima()</code>	Avança a faixa tocando para a próxima da lista
<code>anterior()</code>	Volta a faixa tocando para a anterior da lista
<code>exibirPlaylist()</code>	Lista todas as músicas, destacando a que está tocando

Extra opcional (se sobrar tempo): implementar o **modo repetir**, fazendo a última música apontar de volta para a primeira (e vice-versa), transformando a lista em circular. Não é obrigatório para fechar o projeto em 1h30, mas é um ótimo gancho para a próxima aula sobre listas circulares.

ROTEIRO SUGERIDO (1H30)

1. **(15 min)** Modelar a classe `Musica` (nó) com `titulo` , `artista` , `anterior` e `proximo` .
2. **(20 min)** Implementar `adicionarMusica()` e `exibirPlaylist()` .
3. **(20 min)** Implementar `removerMusica()` , tratando remoção no início, meio e fim.
4. **(20 min)** Implementar `proxima()` e `anterior()` com o ponteiro "tocando".
5. **(15 min)** Testar e, se sobrar tempo, tentar o modo repetir opcional.